LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 Inheritance



NAMA : Wahyuni Salsabila

NIM : 103132400010

KELAS : S1 SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pemrograman berorientasi objek (OOP), objek-objek yang ada di sekitar kita pada dasarnya bisa disusun secara bertingkat atau hierarkis dari yang sifatnya umum ke yang lebih khusus. Dari pola ini muncul konsep pewarisan (*inheritance*) dan perubahan bentuk (*polymorphism*). Keduanya dipakai supaya kita tidak perlu menulis ulang variabel atau *method* yang sama berulang kali di banyak *class* (menghindari duplikasi kode), sekaligus memungkinkan beberapa objek yang masih satu turunan dipanggil lewat *interface* atau nama *method* yang sama meskipun detail proses di dalamnya berbeda.

1. Konsep Inheritance (Pewarisan)

Inheritance merupakan mekanisme di Java untuk menurunkan suatu *class* yang sifatnya masih umum menjadi *class* baru yang lebih spesifik menggunakan kata kunci extends. Hubungan antar *class* ini sering disebut sebagai hubungan *"is-a"* (adalah bagian dari). Di dalam *inheritance*, *class* pemberi warisan disebut sebagai kelas induk (*superclass* atau *base class*), sedangkan *class* yang menerima warisan disebut kelas turunan (*subclass* atau *derived class*). Nantinya, semua karakteristik umum baik berupa atribut (*state*) maupun perilaku (*behaviour/method*) milik *superclass* otomatis dimiliki juga oleh *subclass*-nya.

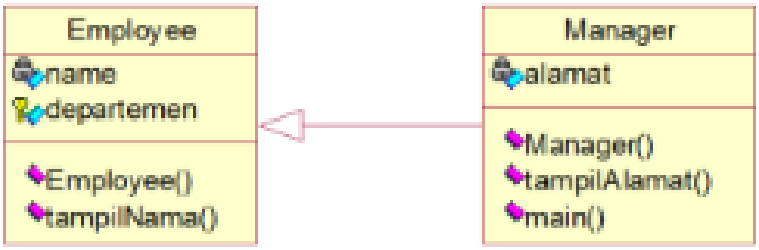


Gambar 6- 1 Ilustrasi pada kelas manusia

Kalau dilihat dari ilustrasi di atas, class manusia berperan sebagai *superclass* yang menyimpan *state* umum (berat, tinggi, usia) serta *behaviour* dasar (makan(), minum(), bekerja()). Ketika diturunkan menjadi class wanita dan class Pria, kedua *subclass* tersebut otomatis mewarisi seluruh atribut dan *method* manusia tadi tanpa harus mendeklarasikannya ulang dari awal, lalu tinggal menambahkan atribut atau perilaku spesifik yang hanya dimiliki oleh masing-masing kelas turunan.

2. **Penggunaan Keyword Super**

Saat bekerja di dalam *subclass*, kita sering kali tetap butuh mengakses komponen yang ada di kelas induknya. Untuk keperluan itu, Java menyediakan kata kunci super yang berfungsi sebagai penunjuk (*reference*) langsung ke *superclass*. Keyword ini umumnya dipakai dalam tiga kondisi: memanggil *constructor* kelas induk lewat super(...) di baris pertama *constructor* anak, mengakses atribut milik kelas induk (super.namaAtribut), atau memanggil *method* milik kelas induk (super.namaMethod()).



Gambar 6-2 Contoh kelas dengan keyword super

Seperti yang terlihat pada diagram UML di atas, Employee merupakan *superclass* dari Manager. Saat objek Manager dibuat, kelas Manager tidak perlu membuat ulang variabel name dan departemen, melainkan cukup memanggil *constructor* milik Employee menggunakan super(nama) untuk mengisi nama, serta memakai super.departemen dan super.tampilNama() ketika ingin mengisi departemen maupun mencetak nama pegawai dari dalam method tampilAlamat().

3. **Overloading Method**

Di dalam satu *class* yang sama, Java memperbolehkan kita membuat dua atau lebih *method* dengan nama yang persis sama, asalkan jumlah parameter atau tipe data parameternya dibuat berbeda satu sama lain. Teknik ini dinamakan *Method Overloading*. Tujuannya supaya sebuah *method* punya fleksibilitas saat dipanggil dengan berbagai variasi input. Misalnya pada kasus pengisian data lagu, kita bisa membuat method IsiParam(String param1) yang hanya menerima judul lagu saja (dengan nama pencipta otomatis diatur menjadi "Tidak dikenal"), dan satu lagi IsiParam(String param1, String param2) yang menerima judul sekaligus nama penciptanya. Saat program dijalankan, Java akan otomatis mencocokkan *method* mana yang harus dieksekusi berdasarkan argumen yang kita kirimkan.

4. **Overriding Method**

   Berbeda dengan *overloading* yang berada di satu *class* dan dibedakan lewat parameter, *Method Overriding* terjadi antar kelas induk dan kelas anak. *Overriding* dilakukan ketika sebuah *subclass* mendefinisikan ulang *method* dengan nama dan susunan parameter yang sama persis seperti yang sudah ada di *superclass*-nya. Ketika *method* tersebut dipanggil melalui objek *subclass*, maka versi *method* milik *subclass*-lah yang akan dijalankan untuk menggantikan perilaku bawaan dari *superclass*. Namun, jika kita masih ingin menjalankan isi perintah asli dari kelas induk sebelum menambahkan proses baru di kelas anak, kita tetap bisa memanggil method induknya terlebih dahulu di bagian dalam method menggunakan super.namaMethod().

B. Guided

I. Guided 01

1. A.java

```
package modul6.caca.guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+" , y= "+y);
    }

}
```

2. B.java

```
package modul6.caca.guided.gd1;

public class B extends A {
    int z;
    int hasil;
    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = "+ hasil);
    }
}
```

3. Demo.java

```java
package modul6.caca.guided.gd1;

public class Demo {
    public static void main(String[] args){
       A superclass = new A();
       B subclass = new B();

       superclass.x =3;
       superclass.y=4;

       System.out.println("Superclass:");
       superclass.tampilXY();

       subclass.x= 1;
       subclass.y= 2;
       subclass.z= 5;

       System.out.println("Subclass:");
       subclass.jumlahXYZ();
    }

}
```

Output :

```
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-
06-inheritance\caca\src\main\java> java modul6.caca.guided.g
d1.Demo
Superclass:
Nilai x = 3 , y= 4
Subclass:
Jumlah (x+y+z) = 8
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-
```

II. Guided 02

- Employee.java

```java
package modul6.caca.guided.gd2;
```

```
public class employee {
    private String name;
    String departemen;

    public employee(String name){
        this.name = name;
    }
    public void tampilNama(){
        System.out.println("Nama: "+name);
    }

}
```

```
package modul6.caca.guided.gd2;

public class employee {
    private String name;
    String departemen;

    public employee(String name){
        this.name = name;
    }
    public void tampilNama(){
        System.out.println("Nama: "+name);
    }

}
```

- Manager.java

```
package modul6.caca.guided.gd2;

public class Manager extends employee {
    private String alamat;

    public Manager(String name, String alamat){ //cara membuat
constructor dalam subclass, harus memanggil constructor
superclass dengan super()
        super(name);
        this.alamat = alamat;
    }
    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("Alamat: "+alamat);
        System.out.println("Departemen: "+super.departemen);
    }
}
```

- Demo.java:

```
package modul6.caca.guided.gd2;

public class Demo {
    public static void main(String[]args){
        Manager adi = new Manager("Adi", "Jl. Merdeka");
        adi.departemen = "IT";
        adi.tampilInformasi();
    }
}
```

Output :

```
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-
06-inheritance\caca\src\main\java> java modul6.caca.guided.g
d2.Demo
Nama: Adi
Alamat: Jl. Merdeka
Departemen: IT
PS D:\praktikumPBO\repo\praktikum-pbo-ps1ds-05-reg-01\modul-
```

III. Guided 03

- **Lagu.java**

```
package modul6.caca.guided.gd3;

public class Lagu {
    String pencipta;
```

```
    String judul;

    void IsiParam(String param1){
        judul = param1;
        pencipta ="Tidak dikenali";
    }
    void IsiParam(String param1, String param2){
        judul = param2;
        pencipta = param2;
    }
    void tampilkan(){
        System.out.println("Judul :"+ judul + ", pencipta:"+
pencipta);
    }

}
```

- **Main.java**

```
package modul6.caca.guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d,e;
        d=new Lagu();
        e=new Lagu();

        d.IsiParam("Lagu1");
        e.IsiParam("Lagu2","aku");
        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
06-inheritance\caca\src\main\java> java modul6.caca.guided.g
d3.Main
Judul :Lagu1, pencipta:Tidak dikenali
Judul :aku, pencipta:aku
```

IV. Guided 04

- **A.java**

```java
package modul6.caca.guided.gd4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A
dipanggil...");
    }

}
```

- **B.java**

```java
package modul6.caca.guided.gd4;

public class B extends A {
    public void tampilkan(){
        System.out.println("Ini adalah class B");
    }

}
```

- **Main.java**

```java
package modul6.caca.guided.gd4;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :

```
n\java> java modul6.caca.guided.gd4.Main
Ini adalah class B
```

C. Unguided

1. Soal Unguided

https://docs.google.com/document/d/1uA5OfXP2VpUzJKFOp35NeGF14Fih9VMA/edit#heading=h.fg49v4603y1o

Jawaban :

- ModelML.java

```
package modul6.caca.unguided;

public class ModelML {
    private String nama;
    private int dataLatih;

    public ModelML(String nama, int dataLatih) {
        this.nama = nama;
        this.dataLatih = dataLatih;
    }

    public void tampilInfo() {
        System.out.println("Model: " + nama + " | Data latih: "
+ dataLatih);
    }

    public int hitungBiaya() {
        return dataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return dataLatih * epoch;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public int getDataLatih() {
        return dataLatih;
    }

    public void setDataLatih(int dataLatih) {
        this.dataLatih = dataLatih;
    }
}
```

- ModelRegresi.java

```
package modul6.caca.unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String nama, int dataLatih, int
jumlahFitur) {
        super(nama, dataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batch) {
        return getDataLatih() * epoch / batch;
    }

    public int getJumlahFitur() {
        return jumlahFitur;
    }

    public void setJumlahFitur(int jumlahFitur) {
        this.jumlahFitur = jumlahFitur;
    }
}
```

- ModelKlasifikasi.java

```
package modul6.caca.unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int dataLatih, int
jumlahKelas) {
        super(nama, dataLatih);
        this.jumlahKelas = jumlahKelas;
    }
```

```
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }

    public int getJumlahKelas() {
        return jumlahKelas;
    }

    public void setJumlahKelas(int jumlahKelas) {
        this.jumlahKelas = jumlahKelas;
    }
}
```

- ModelDeepLearning.java

```
package modul6.caca.unguided;

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String nama, int dataLatih,
int jumlahKelas, int jumlahLayer) {
        super(nama, dataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah layer: " +
jumlahLayer);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }

    public int getJumlahLayer() {
        return jumlahLayer;
    }

    public void setJumlahLayer(int jumlahLayer) {
        this.jumlahLayer = jumlahLayer;
    }
}
```

- Main.java

```
package modul6.caca.unguided;

public class Main {
    public static void main(String[] args) {
        ModelRegresi m1 = new ModelRegresi("Regresi
Linier", 500, 4);
        ModelKlasifikasi m2 = new
ModelKlasifikasi("Decision Tree", 800, 3);
        ModelDeepLearning m3 = new
ModelDeepLearning("Neural Network", 1000, 5, 3);

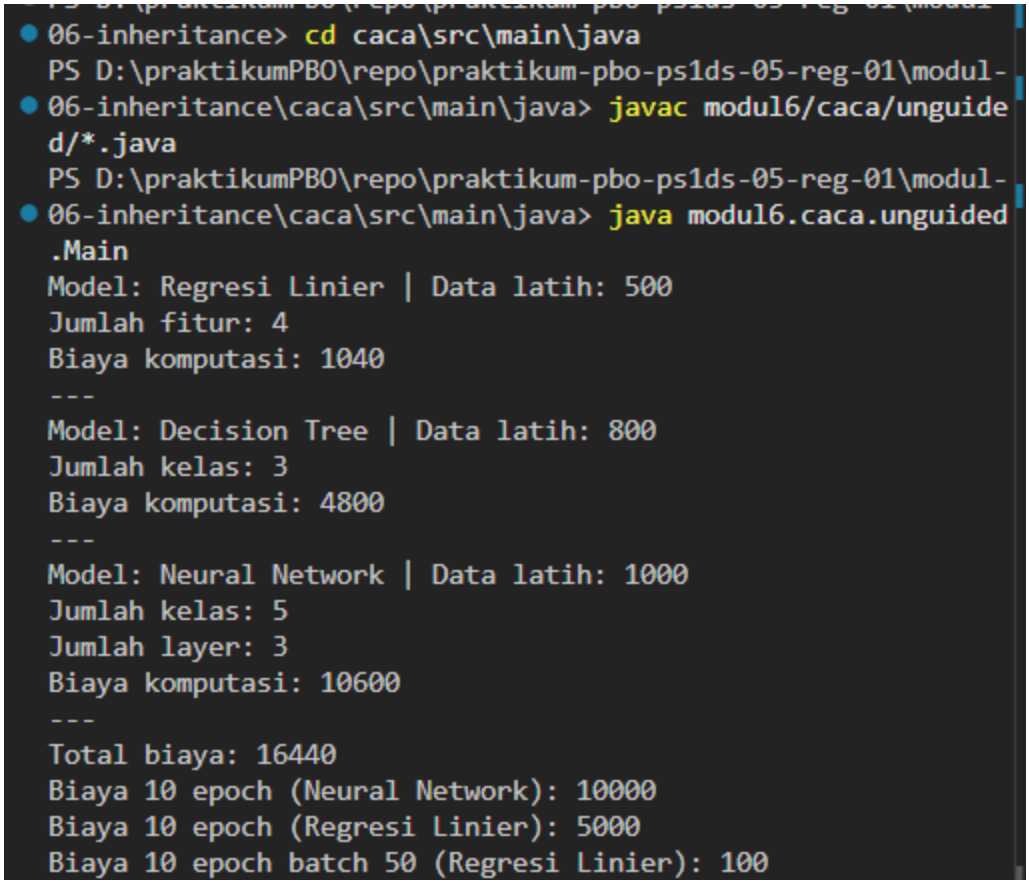
        ModelML[] listModel = new ModelML[3];
        listModel[0] = m1;
        listModel[1] = m2;
        listModel[2] = m3;

        int total = 0;

        for (int i = 0; i < listModel.length; i++) {
            listModel[i].tampilInfo();
            int biaya = listModel[i].hitungBiaya();
            System.out.println("Biaya komputasi: " +
biaya);
            System.out.println("---");
            total = total + biaya;
        }

        System.out.println("Total biaya: " + total);
        System.out.println("Biaya 10 epoch (Neural
Network): " + listModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi
Linier): " + m1.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50
(Regresi Linier): " + m1.hitungBiaya(10, 50));
    }
}
```

Output :



Penjelasan :
Program ini aku buat untuk menghitung dan membandingkan estimasi biaya komputasi dari beberapa jenis model machine learning dengan menerapkan konsep pewarisan (inheritance). Di sini ModelML berperan sebagai class induk yang menyimpan atribut nama dan dataLatih secara private biar datanya nggak bisa diakses sembarangan dari luar, melainkan lewat getter dan setter. Dari class induk tersebut, aku membuat tiga class turunan secara bertingkat, yaitu ModelRegresi yang menambah atribut jumlah fitur, ModelKlasifikasi yang menambah atribut jumlah kelas, serta ModelDeepLearning yang mewarisi ModelKlasifikasi dengan tambahan atribut jumlah layer. Di masing-masing class anak, aku pakai keyword super untuk meneruskan isi constructor dan memanggil perhitungan dasar dari class di atasnya saat melakukan overriding pada method tampilInfo dan hitungBiaya, ditambah penerapan overloading pada method hitungBiaya yang punya variasi parameter kosong, satu parameter epoch, hingga dua parameter epoch dan batch khusus di model regresi.

Alurnya sendiri bermula di class Main, di mana aku menginisialisasi tiga objek model (m1 untuk Regresi Linier, m2 untuk Decision Tree, dan m3 untuk Neural

Network) lengkap dengan nilai atributnya masing-masing. Ketiga objek itu kemudian aku masukkan ke dalam satu array listModel yang bertipe class induk ModelML, lalu dibaca satu per satu menggunakan perulangan for untuk menampilkan detail tiap model, mencetak biaya komputasi dasarnya, sekaligus mengakumulasikannya ke variabel total. Setelah total keseluruhan biaya ditampilkan, di baris paling akhir program akan menguji method overloading tadi dengan menampilkan biaya pelatihan 10 epoch untuk Neural Network yang diambil dari elemen array, serta biaya 10 epoch biasa dan 10 epoch dengan batch 50 dari objek Regresi Linier.

D. Kesimpulan

Berdasarkan praktikum Pemrograman Berorientasi Objek Modul 6 ini, saya dapat memahami secara mendalam konsep pewarisan (*inheritance*), pemanfaatan kata kunci super, serta perbedaan penerapan antara *method overloading* dan *method overriding*. Melalui serangkaian latihan *guided* dan *unguided*, terlihat jelas bagaimana kata kunci extends mempermudah pembuatan hierarki kelas dari *superclass* yang bersifat umum ke *subclass* yang lebih spesifik—baik satu tingkat maupun bertingkat—sehingga atribut dan *method* dasar tidak perlu ditulis ulang di setiap kelas. Selain itu, praktikum ini memperlihatkan peran penting kata kunci super sebagai jembatan bagi kelas turunan untuk memanggil *constructor*, atribut, maupun *method* milik kelas induknya, terutama ketika atribut di kelas induk tetap dijaga keamanannya menggunakan *modifier* private. Penerapan *method overloading* terbukti memberikan fleksibilitas dalam menyediakan beberapa variasi perhitungan pada nama *method* yang sama melalui perbedaan parameter (seperti perhitungan biaya berdasarkan jumlah *epoch* dan ukuran *batch*), sedangkan *method overriding* memungkinkan setiap kelas turunan menyesuaikan kembali perilaku *method* warisan sesuai kebutuhan spesifiknya. Terakhir, kemampuan menyimpan berbagai objek *subclass* ke dalam satu *array* bertipe *superclass* sangat membantu menyederhanakan pengolahan objek secara seragam melalui perulangan pada studi kasus nyata seperti sistem perbandingan estimasi biaya komputasi model *machine learning*.